

7.1 Shared Responsibility: A Field of Tension

Gro Bjerknes

7.1.1 Introduction

Why is it problematic for computer scientists and domain experts to share responsibility for a system development project, even when shared responsibility is a goal for the project?

I will discuss this topic on the basis of experiences from the Florence project¹. First, I present the Florence project and the organization of the project. Next, I discuss ‘shared responsibility’. Thirdly, I tell about three situations in which the dilemma of shared responsibility was visible in some way or another, and finally, I reflect upon our experiences.

My conclusion is that shared responsibility, seen from a computer scientist’s point of view, turns out to be a field of tension between control of the project on the one hand, and participation of domain experts on the other hand.

7.1.2 The Florence project

The Florence project was a research project about the development and use of computer systems in nurses’ daily work. The original goal of the project was to design programming languages, called profession-oriented languages, based on the professional language of occupational groups. The profession-oriented languages were seen as an offensive strategy for occupational groups to keep control over the use of computers in their work, instead of adjusting the work to computer systems designed from a management point of view. Another hope was to avoid adjusting work to computer systems designed for societies different from the Norwegian one.

The project adheres to a research tradition known as *the collective resource strategy*², or alternately as *the critical tradition*³ of system development in Scandinavia. System development, in short, can be said to be concerned about how to build computer systems that fit the organization in which it will be used.

¹ The Florence project was initiated by the system development group, Dept. of Informatics, University of Oslo, and sponsored by The Royal Norwegian Council for Scientific and Industrial Research. The project lasted from 1983 to 1987.

² [Ehn and Kyng, 1987]

³ [Bansler, 1989]

Thus, two important topics are:

1. What does a fit between a use organization and a computer system mean, and
2. How do we carry out system development to ensure that the resulting computer system actually fits the use organization.

The critical tradition in Scandinavia has its basis in the institutionalized conflict between labour and capital. Hence, these topics have been discussed from a worker's point of view, arguing that capital already had resources to develop computer systems according to their needs. The trade unions were regarded as representatives for the workers' collective, and the first projects within the tradition were carried out in close cooperation with trade unions, aiming at developing strategies for the introduction of computers. However, there are many assumptions built into computer technology, and after a while it was considered necessary to question the technology itself. This led to projects where the goal was to show alternatives to existing technology. The Florence project was one of these projects.

Still, the goal of constructing profession-oriented languages was quite abstract. We also found it difficult to construct a programming language on the basis of natural (professional) languages. Therefore the goal was modified to building a computer system for nurses' daily work. With this goal we could still base our work on a profession, nursing, and we could explore different ways of carrying out system development. In order to build computer systems for nurses' daily work, we had two possibilities. We could do it all on our own, in which case we would have to learn a lot about nursing. Or we could leave it to the nurses to decide what kind of computer applications would suit their work. Then we had to teach the nurses how to evaluate computer systems in relation to their work.

According to the Scandinavian research tradition, we chose the latter and became dependent on the nurses. If we say that the computer scientists represented a technical point of view, and the nurses a work setting point of view, the idea was that the nurses should see the technical solution from a work setting point of view. This view of the computer system should be the basis for design. However, the nurses also became dependent on us, because we were able to evaluate the work setting solutions from a technical point of view by adding our general knowledge of the consequences of computerization.

We wanted to reflect the mutual dependency in the organization of the project, by sharing the responsibility for the project with the nurses. The mutual dependency was reflected in the first period of the project, called 'mutual learning'. This period was inspired by anthropological research methods, introduced by the anthropologist that participated in the project. The goal of these activities was that the computer scientists should learn about nursing, and that the nurses should learn about informatics. The 'mutual learning' period ended with negotiations about the application area of the computer system we intended to build, evaluated on both a nursing and an informatical basis.

7.1.3 The notion of 'shared responsibility'

The notion of 'shared responsibility' was seldomly explicitly stated in the Florence project, it was more like an implicit assumption that guided the planning of the project and the way the project was carried out.

The bases for the concept of shared responsibility are the following statements:

- There are different interests in a project, and the differences are a source of knowledge.
- The different groups are equally necessary.
- The responsibility for the project as a whole should be shared between the participants. To share responsibility means both to give and to take responsibility, according to the situation.

The first two statements can be seen as a political credo. The statements express the importance of the domain experts, in this case nurses. The emphasis on domain experts is closely related to the critical tradition in Scandinavian system development research. The statements also stress the importance of computer scientists. Together the statements express the view that even though domain experts are experts on their own work, computer scientists can contribute to how to build a computer system that fits the work. The third statement seems to be a logical consequence of the first two. However, it proved to be very difficult to share responsibility with the nurses. As the anthropologist in the project put it: "Should the users decide on their own, or should they have democratic system developers who decide for them?"

7.1.4 Three situations

The dilemma of shared responsibility was encountered in three situations:

1. when we decided what computer system to build,
2. when we had serious trouble due to bugs in the computer, and
3. when the computer system was introduced in the work place.

The dilemma appeared foremost in situations where the computer scientists wanted to control the process for one reason or another, e.g. keeping the time schedule, protecting research interests, and at the same time it was imperative that the nurses had control as well, in order to reach the overall goal of the project.

In the first situation, the computer scientists wanted to build a computer system that was interesting from a research point of view. On the other hand, the system should also be a system the nurses wanted to use in their daily work, otherwise there would be no research connected to the study of computer systems in use.

In the second situation, the computer scientists excluded the nurses from the development process by giving priority to programming problems instead of using at least some time to keep the nurses in a kind of busy-wait state.

In the third situation, the computer scientists insisted on leaving the instruction and introduction of the computer system to the nurses. Again, we would have liked to exercise a kind of control to ensure that the system was used. However, we had no means to force the nurses to use the system.

Deciding on what computer system to build

We were determined to let the nurses decide what kind of computer applications they wanted. This was mostly due to earlier experiences: it is hard to motivate a sense of cooperation if the nurses do not judge the proposed computer system to be useful for their daily work. We also would insist on building a computer system for the nurses' daily work. We were not interested in making carbon copies of general administrative or medical systems. And both groups wanted to build a computer system that gave a fair chance to succeed with the available resources.

We decided on the computer system by setting up a negotiating meeting. In the week before the meeting, we were rather nervous. Could we really trust that the nurses had sufficient understanding of informatics to propose a useful computer system? What should we do if there were a total disagreement? Should we try to persuade them in some way or another? Should we set up new negotiations? How should we do this?

Fortunately, we did agree on a computer system. We decided to build a "work sheet system", i.e. the computer system should produce work sheets that were to replace the clipboards that the nurses use during a shift. The clipboards contain all relevant information about the patients in the ward, and they are used as reminders and notepads during a shift. The work sheet system was a small and simple system. At that time, we knew enough about nursing to recognize that even if the computer system was simple, it would interfere with the nurses' daily working routines and the information structures they used in many and complex ways. Seen from a system development point of view, this was a challenge. And, most important, we could rely on the nurses. Our anxiety to leave important decisions to the nurses seemed to be unnecessary.

So, we remained in peace with the nurses. Instead, the decision brought up other conflicts with existing traditions:

- We found ourselves in conflict with the assumptions of what researchers should do. Neither in a research milieu, represented by the financing institutions, nor in the technical milieu, represented by the computer manufacturer, is it acceptable to make simple and useful computer systems. Instead, the systems are supposed to be technically advanced, and result in new and fancy products. This is due to the fact that the (research) goal often is meant to support the development of new Norwegian information technology, which seems to be in opposition to building useful computer systems for particular users.
- The computer system did not correspond to the usual way of using computers in hospitals. The usual way is to make computer systems for the

administration of goods, personnel and patients, to make medical systems for medical doctors or to make statistics for planning. Thus, our computer system represented a new way of looking at the use of information in hospitals. Unfortunately, this is difficult to explain to computer scientists, because they do not know anything about hospitals. And for people in health care, the system corresponded to the way they actually use information, therefore they could not see anything new in it. They could not see that the traditional way of using computers in hospitals often is a mismatch with the way information actually is used.

- Due to the deviation from tradition, another conflict was brought to the surface; namely, two opposing views on how to solve the problems in health care: One side prefers technical solutions, like computer-based medical records, the other side prefers organizational solutions. We were lucky since the larger part of our steering group belonged to the 'organizational' side.

This illustrates that we could rely on the nurses to share responsibility, but that the consequences of shared responsibility are not necessarily accepted outside a project.

Bugs in the computer

The project was behind schedule due to hardware errors, software errors and the fact that we had to explore the limits of the computer tools we were using. During the implementation, we worked at the University most of the time, for several reasons: It was a half hour drive each way to the hospital; at the University we had a workplace, with telephone available all the time, a black-board which we used for planning and writing down design decisions, a proper place for manuals and so forth; in the hospital we had one terminal only, and we always worked in rooms that were designed according to the nurses' needs.

What was the result of this? When we did not have the regular contact with the nurses, they lost the possibility for taking responsibility for the project. We excluded them from the process, even if we did not intend to do so. We just thought that they were not interested in programming and debugging. What happened was that they lost motivation; they could not understand why things took so long. They had seen a prototype made with a text-editor, and they could not understand why it took us such a long time to implement the system; nothing seemed to happen in the project. Thus, it was difficult for them to legitimize the project to the other nurses in the ward. The result was almost a total collapse of the project. We rescued it at the last minute.

This illustrates how easy it is to exclude people from taking responsibility even when the goal is the opposite.

Introduction of the computer system

After teaching the members of the project group how to use the system and making short, written instructions, the responsibility for instruction and intro-

duction of the computer system was completely left to the nurses. Then we left, and we did not show up for half a year.

Since the nurses have been responsible for the suggestion of a useful computer system, it was rather easy for us to let them take the responsibility for introducing and using it. After all, it was not necessary for us to prove to the nurses that the computer system was useful to them.

In the beginning, the nurses were a bit reluctant, and could not believe that we really meant to leave them on their own with the system. But when they got used to the idea, it worked fine. As a matter of fact, their way of organizing instruction was much better than anything we would have proposed. One reason for this was that they exploited the existing organization of work.

The nurses are divided into kernel groups that work the same shift. So the instruction was based on a two-step strategy: One person from every kernel group should learn to use the system, and they should teach the two other nurses in their group when time allowed for it. The nurses were also able to exploit the differences in workload between night and day. So they practised a lot during the night and evening shifts. Where we only saw constraints, they saw possibilities.

This illustrates that the results may be better when we dare to hand over responsibility to others. Still, it was tempting to control the nurses and in this way keep the responsibility for the process, especially when things were going slowly.

7.1.5 An attempt at an explanation

In principle, shared responsibility means both giving and taking responsibility. We found that it was difficult for us to give responsibility. Somehow, we felt that it was easy for us to take responsibility, and that responsibility was easily given to us.

One could ask, why was it important to share responsibility with the nurses if it was so difficult? In belated wisdom, we would say that the notion of 'shared responsibility' was our solution to the field of tension⁴ between control and a participative process.

It seems that computer scientists like to have control⁵. However, it is not only computer scientists themselves who expect to control system development; others expect this as well. 'Control' in this setting means that the computer scientists often are in charge of both the computer system and the process of building it. This can prevent user participation in several areas.

- When computer scientists are building a computer system, they want the result to fulfill some quality criteria. To reach the desired level of *use quality*

⁴ 'Field of tension' is a concept which is inspired by Mao Tsetung's concept of contradictions [Tsetung, 1967]. I have chosen the concept of 'fields of tension' because I like the intuitive meaning of this notion.

⁵ Cf. Burstall, Chap. 2.3.

the domain experts⁶ must participate in the process⁷. On the other hand, the domain experts' knowledge does not guarantee a computer system with good use quality. This is the reason why both kinds of expertise are necessary in system development⁸. The need for both kinds of expertise creates uncertainty. It seems impossible to determine whether the domain experts have a sufficient background for evaluating a computer system for their own work, and it seems equally impossible for computer scientists to know in advance whether they will have sufficient knowledge about the application area to evaluate the domain experts' suggestions for computer systems. It is quite tempting for computer scientists to reduce this uncertainty by controlling and dominating the system development process. In fact, it is quite hard not to dominate it.

- Even if the computer scientists are responsible for the process, it is possible to *delegate well defined tasks*, like education. Since they are often supposed to be responsible for the overall time schedule, it can be difficult not to intervene if the activity does not proceed as scheduled. In this way, the feeling of control may destroy the involvement of domain experts, because it is somehow contradictory to delegate the responsibility for a task and then control the people who carry it out.
- The computer scientists also have to design and implement the computer system. Of course they have their own time schedules for those tasks. Often they think that the problem to be solved and the specified requirements are well defined, and they do not expect that they will need the domain experts after this point in time. First, it often proves to be wrong that everything was well understood⁹. Second, the computer scientists do not give priority to keeping in touch with the domain experts due to time constraints. If they have to choose between implementing a specification according to time schedule or to contact the domain experts to confirm that the specification is correct from the domain experts' point of view, the computer scientists usually choose to keep the time schedule. In this way, the efforts of the computer scientists to control their own tasks may prevent the domain experts from participation in the process.

In theory, shared responsibility is the field of tension between the computer scientists' and the domain experts' control of the development process. In reality, however, much control is given to or taken by computer scientists. Thus, shared responsibility will be a field of tension for computer scientists between control and participation of domain experts.

Acknowledgements

I would like to thank Reinhard Keil-Slawik for his constructive comments on this chapter.

⁶ I use the notion 'domain expert' instead of 'user', since the notion of 'user' pinpoints that the most important aspects of people's work is that they use a computer system.

⁷ [Rolskov, 1990]

⁸ [Bjerknes and Bratteteig, 1988]

⁹ [Florence Report, 1985, Sieker and Jensen, 1988]